A Multi-Dimensional Priority Model for Heterogeneous DAG Workloads in Distributed Computing Continuums

Authors
Affiliation

***Abstract*—Distributed meta-orchestration platforms coordinate workloads across heterogeneous infrastructure spanning cloud, edge, and HPC systems. These platforms must schedule diverse workloads—from safety-critical mine seismology to autonomous AI agents to best-effort research analytics—on shared executor pools. Existing schedulers rely on a single integer priority, which conflates urgency, importance, fairness, and domain-specific value into one dimension, leading to priority inversions and starvation.**

We analyze eight realistic workload types—including self-modifying agentic AI workflows and real-time IoT edge scenarios requiring preemption—identify common priority dimensions, and evaluate six scheduling approaches. Based on this analysis, we propose a hybrid priority model combining lexicographic urgency classes with weighted multi-dimensional scoring within each class. The model supports static, dynamic, and DAG-dependent priority dimensions while providing structural guarantees for safety-critical workloads, preemption on resource-constrained executors, and multi-tenant fairness.

***Index Terms*—distributed scheduling, DAG workflows, priority models, computing continuums, meta-orchestration, agentic AI, preemption**

I. Introduction

Computing continuums [1] integrate heterogeneous resources—cloud data centers, HPC clusters, edge devices, and IoT infrastructure—into unified execution environments. Meta-orchestration platforms such as ColonyOS [2] coordinate workloads across these environments by acting as brokers between users who declare *what* to compute and distributed executors that know *how* to execute on their specific platforms.

Workloads in these systems are expressed as directed acyclic graphs (DAGs) of processes, where each process specifies a function to execute, its arguments, resource requirements, and dependencies on other processes. The scheduler assigns waiting processes to available executors based on priority.

The fundamental challenge is that these platforms serve radically different workloads concurrently: safety-critical mine seismology, satellite surveillance, *agentic AI systems that dynamically modify their own DAGs*, multi-tenant ETL pipelines with SLA deadlines, infrastructure automation, cross-environment HPC training, large-scale parameter sweeps, and real-time IoT edge systems requiring preemption. Each workload has its own notion of what "important" means, yet they all compete for shared executor resources.

Current systems assign priority as a single integer at submission time. This approach fails in three fundamental ways:

1) **Priority is multi-dimensional.** A seismic event's importance depends on sensor count, mine region, and event classification—not a single number. An AI agent's priority depends on its iteration depth, token budget consumption, and goal criticality.
2) **Priority is dynamic.** An ETL pipeline's urgency increases as its SLA deadline approaches. An agentic workflow's resource profile is unknown at submission—it depends on LLM decisions at runtime. A drug screening compound's priority depends on docking results from an earlier DAG step.
3) **Priority is contextual.** A cloud burst workflow's urgency depends on another workflow's queue depth. On resource-constrained edge devices, an emergency anomaly response must preempt running lower-priority monitoring processes.

In this paper, we make the following contributions:

- We present eight realistic use cases spanning the diversity of workloads in computing continuums, including self-modifying agentic AI workflows and real-time IoT with preemption (Section III).
- We extract common priority dimensions and identify six requirements that any priority model must satisfy (Section IV).
- We evaluate six alternative scheduling approaches against these requirements (Section V).
- We propose a hybrid priority model combining lexicographic urgency classes with weighted multi-dimensional scoring, and show how it instantiates for each use case (Section VI).

II. Background

A. Computing Continuums and Meta-Orchestration

A computing continuum provides a unified abstraction over heterogeneous infrastructure. Unlike traditional orchestrators tied to specific platforms (Kubernetes for cloud, SLURM for HPC), a meta-orchestrator operates *above* platform-specific schedulers, coordinating workloads across environments without directly controlling the underlying resources.

In ColonyOS, the core abstractions are:

- **Colony:** A distributed runtime environment comprising networked executors.

- **Executor:** A worker process that connects to the colony from any network location, pulls process assignments, executes them, and reports results. Executors can run on cloud VMs, HPC nodes, edge devices, or any Internet-connected system.
- **Process:** A unit of work defined by a function specification, transitioning through states: WAITING → RUNNING → SUCCESS or FAILED.
- **ProcessGraph:** A DAG of processes with dependency edges. Child processes wait until all parent processes complete successfully.
- **WorkflowSpec:** A declarative specification of a ProcessGraph, defining processes and their dependencies by name.

Executors request work by calling `Assign()`, which returns the highest-priority waiting process matching the executor's type and capabilities. This pull-based model means the scheduler's priority function directly determines which work gets done first.

B. DAG Scheduling

DAG scheduling is well-studied in parallel computing [3]. Classical algorithms (HEFT [4], CPOP) focus on minimizing makespan on a fixed set of processors. Our setting differs in several ways: executors are heterogeneous and dynamically available, DAGs can be modified at runtime (processes inserted between existing nodes), and multiple independent DAGs compete for shared resources with different urgency levels.

C. Current Priority Mechanism

In the current system, each process has an integer `Priority` field set at submission time. The scheduler computes a `PriorityTime` value:

$$\text{PriorityTime} = \text{Priority} \times \Delta t + \text{SubmissionTime} \quad (1)$$

where Δt is a constant scaling factor. The process with the lowest PriorityTime is assigned first. This creates a FIFO ordering within the same priority level, with higher-priority processes jumping ahead. While simple, this mechanism cannot express the multi-dimensional, dynamic, context-dependent priority requirements we identify in our use cases.

D. Dynamic DAG Modification

ColonyOS supports runtime modification of ProcessGraphs through the `AddChild(insert=true)` mechanism. This allows an executor to insert new processes *between* existing nodes in a running DAG, effectively growing the workflow at runtime. This capability is central to the agentic AI use case (Section III-C), where an LLM-powered agent iteratively expands its plan, and distinguishes our setting from classical DAG schedulers that assume a fixed graph structure.

III. Use Cases

We present eight use cases that collectively represent the diversity of workloads in computing continuums. Each is implemented as a working Go program that submits ProcessGraphs to a ColonyOS instance. Table I summarizes the key characteristics and priority patterns.

A. UC1: Mine Seismology (Continuous Stream)

A seismic monitoring network at the LKAB Kiruna iron ore mine has 12 geophones at four underground levels (1165–1450m depth). Seismogram data streams via RabbitMQ; a new processing DAG is submitted every 60 seconds with three stages: (1) **P-wave picking** (12 parallel)—STA/LTA detection with AIC onset refinement on each geophone; (2) **Event classification** (1 process)—a CNN classifier groups picks into events (blast, rockburst, noise); (3) **3D location** (N parallel)—grid search with Simplex refinement using a layered velocity model. Each DAG has $12 + 1 + N$ processes.

Priority: Rockbursts must preempt blasts (safety-critical). Events with more triggered sensors indicate larger magnitudes and should be prioritized—but sensor count is only known *after* classification (runtime-dependent). Active production levels are higher priority than exploration zones.

B. UC2: Satellite Surveillance (Event-Triggered, Multi-User)

Multiple organizations define surveillance jobs monitoring geographic areas using Sentinel-2 imagery. When new imagery is available, a 4-step DAG is spawned: download → preprocess (cloud mask, co-register) → change detection → notify. Five concurrent jobs from Swedish agencies cover illegal logging, highway construction, flooding, wildfire recovery, and coastline erosion, each using domain-specific algorithms (BFAST Monitor, coherence change, MNDWI, dNBR, NDWI+Otsu).

Priority: Detection type determines urgency (wildfire $>$ flooding $\gg$ erosion monitoring). Multiple jobs trigger simultaneously from the same satellite pass. Cross-organization fairness is required.

C. UC3: Agentic AI Workflows (Self-Modifying DAG)

Users submit natural-language goals to an LLM-powered agentic executor [2]. The system constructs and *iteratively extends* a DAG at runtime:

1) A skeleton DAG is submitted: `agent_query` → `llm_plan` → `plan_handler` → `completed`.
2) The plan handler parses the LLM's output and *inserts* tool steps and a reflect node into the DAG between existing nodes using `AddChild(insert=true)`.
3) After tool steps execute, the reflect node evaluates results. If more work is needed, additional steps and another reflect are inserted.
4) This loop continues until the goal is achieved or a resource limit is reached.

This produces a *self-modifying DAG*: the workflow decides its own shape at runtime (Fig. 1). Each iteration adds 2–5 processes, and typical workflows run 3–10 iterations, producing DAGs of 10–50 processes from a 4-process skeleton.

TABLE I
SUMMARY OF USE CASES AND THEIR DOMINANT PRIORITY CHARACTERISTICS.

UC	Domain	DAG Pattern	Dominant Priority Dimension	Temporal Class	Infrastructure
1	Mine seismology	Continuous stream, fan-out/fan-in	Event magnitude, mine region	Runtime-dependent	On-premise
2	Satellite surveillance	Event-triggered, multi-user	Detection type urgency	Static	Cloud
3	Agentic AI	Self-modifying, iterative growth	Iteration depth, token budget	Runtime-dependent	Cloud
4	Multi-tenant ETL	SLA-tiered parallel pipelines	Deadline proximity, SLA tier	Dynamic	Cloud
5	Cloud burst	Meta-level infrastructure	External queue depth	Externally-dependent	Cloud
6	HPC ML training	Cross-environment, parallel tracks	Environment transfer latency	Externally-dependent	Cloud + HPC
7	Drug discovery	Deadline-driven funnel	Binding affinity, deadline	Runtime + dynamic	HPC (GPU)
8	Water treatment IoT	Mixed criticality, preemptive	Anomaly severity, latency budget	Static + dynamic	Edge

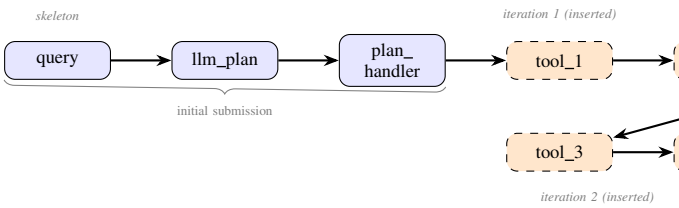

Fig. 1. Agentic AI DAG growth. Solid nodes are the initial skeleton; dashed nodes are dynamically inserted via `AddChild(insert=true)`. The DAG shape is determined by LLM decisions at runtime.

Multiple users submit goals concurrently, each producing an independent ProcessGraph that grows dynamically and unpredictably. Three representative workloads run simultaneously: a data science agent analyzing soil moisture datasets, a DevOps agent writing an MQTT-to-InfluxDB integration, and a student agent requesting explanations of scheduling concepts.

Priority requirements:

- **Unknown DAG size:** Resource consumption is unpredictable at submission time—a simple question may resolve in one iteration, while a complex coding task may require ten.
- **Iteration depth decay:** An agent on its 15th iteration is consuming shared resources with potentially diminishing returns. Priority should decay with iteration depth to prevent runaway agents from monopolizing executors.
- **Token/cost budget:** LLM API calls are expensive. An agent approaching its token budget limit may warrant priority adjustment—either boosted (finish before cutoff) or reduced (diminishing value).
- **Goal criticality:** A production agent handling a customer request should outprioritize an exploratory research agent, but this is a user-declared dimension, not derivable from the DAG.
- **Tool contention:** When multiple agents need the same external tool (database query, API call), priority determines which agent proceeds first.
- **Implicit priority propagation:** Prioritizing an agent's current step implicitly prioritizes all future work it will generate. Unlike fixed DAGs, where the scheduler can see all downstream processes, an agentic DAG's future is opaque.

D. UC4: Multi-Tenant ETL (SLA-Driven)

A shared data platform serves three organizations with different SLAs: **Gold** (5-min deadline, bank fraud detection, 10 processes), **Silver** (60-min, logistics route optimization, 7 processes), and **Bronze** (24h, university bibliometrics, 5 processes). All pipelines compete for the same `etl-worker` executors.

Priority: SLA tier determines base priority. Urgency escalates as deadlines approach. Bronze must not be starved—fairness across tenants is required despite priority differences.

E. UC5: Cloud Burst (Meta-Level Infrastructure)

When the seismic queue (UC1) exceeds on-premise capacity, a cloud burst DAG is triggered: assess queue → estimate capacity → provision VMs (Hetzner) → install K3s → deploy executors + transfer data (ColonyFS) → start → monitor costs → monitor queue → teardown. Budget cap: EUR 50.

Priority: Derives from *another workflow's* queue state (externally-dependent). Creates a feedback loop: burst capacity drains the queue, reducing its own urgency. Budget constraints impose a ceiling independent of priority.

F. UC6: Cloud-to-HPC LLM Training (Cross-Environment)

A cloud platform user fine-tunes Llama 3.1 8B for Swedish legal text. Two parallel preparation tracks (data: validate → tokenize → upload; model: download → convert Megatron → upload) converge at HPC training on Dardel (2 nodes × 4×A100, TP=4, PP=2). Results transfer back via ColonyFS to MLflow registry and vLLM deployment.

Priority: The DAG crosses environment boundaries with long data transfers. HPC processes may queue in SLURM for hours. Priority must span two independent scheduling domains.

G. UC7: Drug Discovery Parameter Sweep (Deadline-Driven Funnel)

A pharmaceutical company screens 25 drug candidates against SARS-CoV-2 Mpro. Phase 1: prepare target → dock all 25 (AutoDock Vina) → 25 short MD simulations (10ns, GROMACS) → rank. Phase 2: top 3 candidates get 100ns extended MD → regulatory report. Hard 48-hour deadline.

Priority: Compounds are prioritized by ML-predicted binding affinity (better candidates get GPU time first). Phase 2

processes outprioritize remaining phase 1. Deadline proximity escalates urgency.

H. UC8: Water Treatment IoT (Real-Time Edge with Preemption)

A water treatment plant (Norrvatten, northern Stockholm) monitors quality across four stages using edge gateways with 1–2 executor slots each. Normal monitoring DAGs (collect → validate → aggregate → dashboard) run every 30s, producing 16 normal-priority processes per cycle. When a sensor breaches a threshold (chlorine < 0.2 mg/L, turbidity > 4 NTU), an emergency DAG is submitted: emergency collect → classify severity → actuate response + alert operators (parallel) → notify SCADA → log incident.

Priority: Emergency processes must *preempt* running normal-priority processes—not merely jump the queue—because edge gateways lack spare capacity. Hard real-time constraint: anomaly to actuator within 2 seconds. Mixed criticality levels share the same resource-constrained executors.

IV. Analysis of Priority Dimensions

Examining the eight use cases reveals that priority is determined by multiple dimensions. We classify these along two axes: *when* the value is known and *where* it comes from.

A. Temporal Classification

Static dimensions have values fixed at submission time: SLA tier (UC4), detection type (UC2), compound predicted binding affinity (UC7), anomaly severity (UC8), user tier and goal criticality (UC3).

Dynamic dimensions change over time without new information: deadline proximity (UC4, UC7), process wait time (all), latency budget remaining (UC8), token budget consumption (UC3).

Runtime-dependent dimensions only become known when intermediate DAG steps complete: triggered sensor count (UC1, after classification), candidate ranking (UC7, after short MD), DAG size and iteration depth (UC3, determined by LLM at runtime), confirmed severity after cross-validation (UC8).

Externally-dependent dimensions derive from state outside the DAG: queue depth of another workload (UC5), HPC queue wait time (UC6), budget remaining (UC5), sensor readings breaching thresholds (UC8).

B. Source Classification

Process-intrinsic: Values from the process's own specification—function name, arguments, resource requirements, executor type.

DAG-structural: Values from the process's position in its DAG—depth, number of downstream dependents, DAG completion percentage, iteration depth in agentic loops (UC3).

Parent-derived: Values from the output of parent processes—classifier results, intermediate scores, LLM plan decisions (UC3).

System-state: Values from the scheduling system—queue depths, executor utilization, elapsed time.

User/tenant metadata: Values from the process submitter—SLA tier, organizational role, budget, user tier.

C. Requirements

From the use cases, we derive six requirements for a priority model:

- **R1 Structural safety guarantees.** Safety-critical work (UC1 rockbursts, UC2 wildfire alerts, UC8 anomaly response) must be guaranteed to preempt non-critical work, independent of parameter tuning.
- **R2 Multi-dimensional scoring.** Priority must be expressible as a function of multiple domain-specific dimensions (sensor count, mine region, event type, compound affinity, anomaly severity, agent iteration depth).
- **R3 Dynamic re-evaluation.** Priority must be recalculable when parent processes complete (UC1), deadlines approach (UC4), external state changes (UC5), or DAGs grow dynamically (UC3).
- **R4 Multi-tenant fairness.** Lower-priority tenants must receive a minimum guaranteed share of resources, preventing starvation (UC4 Bronze tier, UC3 student agents).
- **R5 Interpretability.** Scheduling decisions must be explainable—critical for safety-critical systems (UC1, UC8), SLA compliance auditing (UC4), and understanding why an AI agent was deprioritized (UC3).
- **R6 Preemption support.** On resource-constrained executors (UC8 edge gateways), higher-priority processes must be able to preempt running lower-priority processes, not merely jump the queue.

V. Alternative Scheduling Methods

We evaluate six approaches against the requirements.

A. Weighted Sum Model

The priority of a process p at time t is:

$$\pi(p,t) = \beta(p) + \sum_{i=1}^{n} w_i \cdot d_i(p, G, S, t) \tag{2}$$

where $\beta(p)$ is a base priority, d_i are dimension functions, and w_i are user-configured weights.

Strengths: Simple, interpretable, each dimension independently tunable, $O(n)$ per evaluation. **Weaknesses:** Assumes dimension independence. Dimensions are commensurable: with misconfigured weights, a Bronze process can outprioritize a rockburst alert, violating R1. **Assessment:** R2, R3, R5. Partially R4. Fails R1, R6.

B. Rule-Based / Production Systems

An ordered set of condition-action rules; the first match determines priority. **Strengths:** Handles non-linear dimension interactions; readable and auditable (R1, R5). **Weaknesses:** Rule explosion with growing dimensions. Dynamic rules require complex meta-rules. **Assessment:** R1, R5. Partially R2. Fails R3, R4, R6.

C. Lexicographic / Hierarchical Priority

Priority is a tuple $\pi(p) = (c_1(p), c_2(p), \ldots, c_k(p))$ compared left to right. **Strengths:** Safety is structural—if urgency class is c_1, safety-critical work *cannot* be outprioritized (R1). Used in Linux CFS, Kubernetes PriorityClasses [12]. **Weaknesses:** Rigid within levels; cannot express smooth trade-offs. **Assessment:** R1, R5. Partially R2, R3, R6.

D. Constraint-Based Optimization

Scheduling formulated as: $\underset{x}{\text{maximize}} \sum_{p \in P} U(p) \cdot x_p$ subject to resource, fairness, and deadline constraints. **Strengths:** Most expressive—can encode R1–R6 as constraints. **Weaknesses:** ILP at every scheduling decision does not scale. Requires global view of all pending work. Decisions are opaque (R5 violated). **Assessment:** R1–R4, R6 in theory. Fails R5. Impractical at scheduling-time granularity.

E. Multi-Level Feedback Queue (MLFQ)

Processes start in the highest queue and demote after consuming their time quantum. Used in OS schedulers. **Strengths:** Naturally penalizes long-running processes, providing fairness (R4) without explicit configuration. Handles the agentic AI case: agents that consume many iterations naturally demote. **Weaknesses:** Queue levels are a coarse proxy for priority—cannot express domain-specific dimensions (R2 poorly satisfied). Gaming-susceptible (processes can yield to avoid demotion). Safety guarantees require a reserved top queue with admission control. **Assessment:** Partially R1, R4. Fails R2, R3, R5, R6.

F. Machine Learning Methods

RL: Agents learn scheduling policies by optimizing reward functions (DeepRM [7], Decima [8]). **GNNs:** Encode DAG topology to prioritize bottleneck processes. **Learned weights:** Automate w_i tuning via supervised learning on historical outcomes. **Strengths:** Can discover non-obvious strategies; GNNs handle DAG-aware scheduling; adapt to changing workloads. **Weaknesses:** When a rockburst alert is deprioritized, "the neural network assigned 0.73" is not an acceptable explanation. Training data requirements. Generalization failures on new workload types. **Assessment:** R2–R4. Fails R1, R5, R6.

G. Comparison

Table II summarizes the assessment. No single method satisfies all six requirements.

VI. Proposed Model

We propose a two-level priority model: lexicographic *urgency classes* provide structural guarantees (R1, R5), while a weighted multi-dimensional *scoring function* within each class provides expressiveness (R2, R3). Fairness constraints (R4) are enforced orthogonally. Preemption (R6) is triggered by urgency class transitions. Fig. 2 illustrates the model architecture.

TABLE II
METHOD COMPARISON. ● = SATISFIES, ◐ = PARTIALLY, ○ = DOES NOT SATISFY.

Method	R1	R2	R3	R4	R5	R6
Weighted Sum	○	●	●	◐	●	○
Rule-Based	●	◐	◐	○	●	○
Lexicographic	●	◐	◐	○	●	◐
MLFQ	◐	○	◐	●	○	○
Optimization	●	●	●	●	○	◐
ML (RL/GNN)	○	●	●	◐	○	○
Hybrid	●	●	●	●	●	●

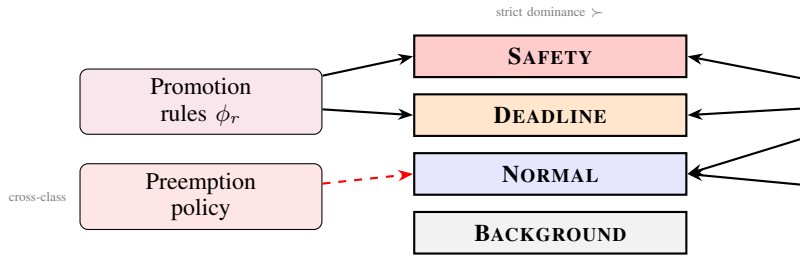

Fig. 2. Priority model architecture. Urgency classes (left) are strictly ordered. Promotion rules assign processes to classes. Within each class, a weighted scoring function orders processes. Fairness constraints prevent starvation. Preemption is triggered across class boundaries.

A. Urgency Classes

Every process is assigned to exactly one urgency class $C \in \{\text{SAFETY}, \text{DEADLINE}, \text{NORMAL}, \text{BACKGROUND}\}$, ordered by strict dominance:

$$\text{SAFETY} \succ \text{DEADLINE} \succ \text{NORMAL} \succ \text{BACKGROUND} \quad (3)$$

A process in a higher class is *always* scheduled before a process in a lower class. Class assignment uses promotion rules:

$$C(p, t) = \max_{r \in R} \{c_r \mid \phi_r(p, G, S, t) = \text{true}\} \quad (4)$$

where R is the set of promotion rules, c_r is the target class, and ϕ_r is a boolean predicate. The default class is NORMAL. Examples:

- SAFETY: `event_type = ``rockburst''` $\wedge$ `sensor_count` $\geq$ `6` (UC1); `anomaly_severity = ``critical''` (UC8).
- DEADLINE: `deadline_remaining` $<$ `0.1` $\times$ `deadline_total` (UC4, UC7); `latency_budget` $<$ `500ms` (UC8).
- BACKGROUND: `sla = ``Bronze''` $\wedge$ `no_deadline` (UC4); `agent_iteration` $>$ `10` $\wedge$ `tier = ``student''` (UC3).

These rules are *auditable*: a human can inspect the rule set and verify that safety-critical work will always be classified correctly. This is a structural guarantee, not a statistical one.

B. Within-Class Scoring

Within each urgency class, processes are ordered by:

$$\sigma(p,t) = \beta(p) + \sum_{i=1}^{n} w_i \cdot d_i(p, G, S, t) \tag{5}$$

where d_i are *dimension functions* mapping a process and its context to a normalized value in $[0,1]$:

- $d_k^{\text{kwarg}}(p) = f(\text{kwarg}(p,k))$: From process arguments.
- $d_k^{\text{parent}}(p,G) = f(\text{output}(\text{parent}(p,G),k))$: From parent output (available after parent completes).
- $d^{\text{dag}}(p,G) = |\{q \in G : \text{state}(q) = \text{SUCCESS}\}|/|G|$: DAG completion fraction ("momentum").
- $d^{\text{wait}}(p,t) = \min(1, (t - t_{\text{submit}}(p))/\tau)$: Normalized wait time (anti-starvation).
- $d^{\text{deadline}}(p,t) = 1 - \min(1, (t_{\text{deadline}}(p) - t)/t_{\text{total}}(p))$: Deadline proximity.
- $d_k^{\text{system}}(S) = f(\text{state}(S,k))$: System state (queue depth, budget).
- $d^{\text{iter}}(p,G)$: Iteration depth in an agentic loop—the number of reflect-plan-execute cycles completed. Used with a negative weight for priority decay.

The transform function f normalizes raw values to $[0,1]$ using linear, categorical, exponential (for deadlines), or logarithmic (for anti-starvation) mappings.

C. Re-evaluation Triggers

The effective priority $\Pi(p,t) = (C(p,t), \sigma(p,t))$ is re-evaluated on:

1) **Parent completion:** Enables parent-derived dimensions and can trigger class promotion (UC1: classification reveals rockburst; UC3: LLM plan determines next steps).
2) **Timer tick:** Updates time-dependent dimensions (d^{wait}, d^{deadline}) and can trigger deadline-based promotion.
3) **System state change:** Queue depth or executor availability changes trigger re-evaluation of d^{system}.
4) **DAG modification:** When new processes are inserted (UC3), the iteration depth and DAG size dimensions are updated for all affected processes.

D. Fairness Constraints

Fairness is enforced orthogonally to priority scoring:

$$\forall j \in \text{Tenants} : \frac{\text{assigned}(j, \Delta t)}{\text{total_assigned}(\Delta t)} \geq s_j \tag{6}$$

When a tenant falls below its minimum share s_j, its next process is scheduled regardless of score. This is particularly relevant for UC3 (agentic AI) and UC4 (multi-tenant ETL), where high-priority agents or Gold-tier pipelines could otherwise monopolize executors.

E. Preemption

When a process is promoted to SAFETY and the target executor has no idle slots, the scheduler preempts the lowest-priority running process. Preemption is:

- Only triggered *across* class boundaries—never within the same class.
- Configurable per executor type: **preemptive** (edge gateways, UC8), **non-preemptive** (cloud/HPC with spare capacity), or **cooperative** (checkpoint before yielding).

The preempted process is marked WAITING with a `preempted` flag and resumed (from checkpoint or restart) when capacity is available.

F. Effective Priority

The complete priority is a tuple compared lexicographically:

$$\Pi(p,t) = \big(C(p,t),\ \sigma(p,t)\big) \tag{7}$$

The scheduler selects the waiting process with the highest Π matching the requesting executor's type.

G. Instantiation Examples

We illustrate model instantiation for four use cases.

1) UC1: Mine Seismology.

Listing 1. Seismic processing priority.

```
promotion_rules:
  SAFETY: event_type="rockburst" AND count>=6
dimensions:
  sensor_count: parent_output("classify","count
      ")
    linear(3,12), weight=5
  mine_region: kwarg("region")
    categorical(production:1, development:0.7),
        w=3
  event_type: parent_output("classify","class")
    categorical(rockburst:1, blast:0.5), w=4
```

The `sensor_count` and `event_type` dimensions are parent-derived: available only after classification completes, triggering re-evaluation and potential SAFETY promotion.

2) UC3: Agentic AI.

Listing 2. Agentic AI priority with iteration decay.

```
promotion_rules:
  BACKGROUND: iteration>10 AND tier="student"
dimensions:
  goal_criticality: kwarg("goal_tier")
    categorical(production:1, research:0.5,
              student:0.2), weight=6
  iteration_depth: dag_iteration_count()
    linear(0,15), weight=-4 # negative = decay
  token_budget_remaining: kwarg("budget_pct")
    linear(0,100), weight=2
  dag_momentum: dag_completion()
    linear(0,1), weight=3
fairness:
  min_share: {production:0.5, research:0.3,
            student:0.1}
```

The *negative* weight on `iteration_depth` causes priority to decay as the agent iterates, preventing runaway agents from monopolizing resources. The `token_budget_remaining` dimension gives a slight boost to agents with budget left (they can still produce useful

work). The BACKGROUND promotion rule demotes student agents that exceed 10 iterations, ensuring they don't block production workloads.

3) UC4: Multi-Tenant ETL.

Listing 3. Multi-tenant ETL priority.

```
promotion_rules:
  DEADLINE: deadline_remaining < 0.1*total
dimensions:
  sla_tier: kwarg("sla")
    categorical(Gold:1, Silver:0.5, Bronze:0.15)
        ,
    weight=15
  deadline_urgency: deadline_proximity()
    exponential(halflife=300s), weight=20
  wait_time: wait_time()
    logarithmic(max=3600s), weight=2
fairness:
  min_share: {Gold:0.50, Silver:0.30, Bronze
      :0.10}
```

4) UC8: Water Treatment IoT.

Listing 4. IoT edge priority with preemption.

```
promotion_rules:
  SAFETY: anomaly_severity = "critical"
  DEADLINE: latency_budget < 500ms
dimensions:
  anomaly_severity: kwarg("severity")
    categorical(critical:1, warning:0.6), w=10
  sensor_deviation: kwarg("value")/kwarg("
      threshold")
    linear(1,5), weight=4
  latency_remaining: latency_budget()
    exponential(halflife=500ms), weight=8
preemption:
  executor_type: edge-iot
  policy: preemptive
  preempt_classes: [NORMAL, BACKGROUND]
```

VII. DISCUSSION

A. The Agentic AI Scheduling Challenge

Use Case 3 introduces a scheduling problem qualitatively different from the other seven: the workload *decides its own shape at runtime*. In all other use cases, the scheduler can inspect the full DAG at submission time (or after a fixed growth phase). An agentic DAG is opaque—prioritizing an agent's current process implicitly prioritizes all future work it will generate, which is unknowable.

This creates a tension between two principles: (1) *sunk cost protection*—an agent that has completed 8 successful iterations has generated value and should be allowed to finish; and (2) *diminishing returns*—an agent that has iterated 15 times without completion is likely stuck and should yield to other workloads. Our model handles this through the iteration depth dimension with a negative weight (priority decay) combined with DAG momentum (sunk cost protection). The balance between these two forces is configurable via weights.

More broadly, the rise of agentic AI workloads challenges the assumption that DAG scheduling is about *fixed graphs*. Future scheduling models must account for self-modifying DAGs as a first-class concept, including resource budgets that bound total agent consumption regardless of priority.

B. Composability

Priority models compose naturally when multiple workload types share a colony. Each workload type defines its own dimensions and weights, but the urgency classes provide a shared framework for inter-workload ordering. A colony administrator defines promotion rules and fairness constraints; workload developers define within-class scoring.

C. Relationship to Classical Scheduling

The model subsumes classical policies as special cases: **FIFO** (no dimensions, equal base priority), **EDF** (one deadline dimension), **SJF** (one dimension on estimated execution time), **Fair Share** (fairness constraints with equal shares), **Fixed Priority with Preemption** (urgency classes with preemptive policy). This generality suggests the model can serve as a unifying framework.

D. Role of Machine Learning

While we argued against ML as a standalone approach, ML complements the model in two ways: (1) **Weight tuning**—learning w_i from historical outcomes while preserving the interpretable structure; (2) **Dimension prediction**—predicting values not yet available (e.g., likely sensor count before classification completes in UC1, or expected iteration count for an agentic workflow in UC3).

E. Preemption Trade-offs

UC8 demonstrates that queue-ordering is insufficient on resource-constrained executors. Preemption introduces costs (restart overhead, checkpointing), but these are acceptable when the alternative violates hard real-time safety constraints. Limiting preemption to cross-class boundaries bounds its frequency. A key open question: if a process in a 90%-complete DAG is preempted, should the DAG momentum bonus apply when rescheduled? We argue yes.

F. Inter-Workflow Dependencies

UC5 illustrates a pattern where one workflow's priority derives from another's state. Our model handles this through d^{system} dimensions, but does not formally model inter-DAG edges. Priority propagation across DAG boundaries—where a high-priority Workflow B causes Workflow A's blocking process to inherit urgency—is an open problem that requires preventing circular dependencies and unbounded propagation.

G. Failure Recovery

A failed process in a partially completed DAG presents a priority dilemma: boost (protect DAG investment), reduce (may fail again), or escalate (approaching deadline). The model's re-evaluation mechanism handles deadline escalation naturally. A *retry count* dimension could address the boost/reduce trade-off, with deployment-specific weight polarity.

H. Limitations

The model assumes urgency classes can be cleanly separated. In practice, class boundaries may be politically contested. The model requires explicit configuration (promotion rules, dimensions, weights, fairness shares), imposing operational burden. Not all workloads are preemptible—GPU kernels and database transactions cannot be cleanly interrupted. The per-executor-type preemption policy mitigates this but requires correct classification by operators.

For agentic AI workloads specifically, the iteration depth dimension with a negative weight is a heuristic. Some agents legitimately require many iterations (complex coding tasks), and a fixed decay penalizes them. An adaptive decay based on progress indicators (e.g., fraction of subtasks completed) would be more principled but harder to configure.

VIII. Related Work

DAG scheduling. HEFT [4] and CPOP prioritize tasks by upward rank for single-DAG makespan optimization on fixed processors. Our work addresses multi-DAG scheduling on dynamic executors with heterogeneous priority semantics.

Multi-criteria scheduling. Braun et al. [5] compare heuristics for heterogeneous computing. Most multi-criteria approaches target makespan and utilization; we focus on priority as a user-facing concern with safety and fairness requirements.

Fair scheduling. DRF [6] provides max-min fairness for multi-resource environments. YARN [9] implements capacity scheduling with hierarchical queues and preemption. Our fairness constraints are simpler but interact with a richer priority model.

Mixed-criticality systems. Vestal [10] introduced mixed-criticality scheduling for real-time systems, where tasks have different criticality levels and the system must guarantee timing for the highest level. Burns and Davis [11] survey the field. Our urgency classes are analogous to criticality levels, extended to DAG workflows in distributed settings rather than periodic tasks on uniprocessors.

Kubernetes scheduling. Kubernetes PriorityClasses [12] implement preemption based on priority levels. Our model extends this with multi-dimensional scoring within each level and DAG-aware re-evaluation.

RL for scheduling. DeepRM [7] and Decima [8] apply deep RL to resource management and DAG scheduling. These sacrifice interpretability, which we argue is essential for safety-critical deployments.

Workflow systems. Apache Airflow, Prefect, and Dagster support DAG workflows with basic priority. None provide multi-dimensional dynamic priority, DAG-aware re-evaluation, or runtime DAG modification.

Agentic AI orchestration. LangGraph, CrewAI, and AutoGen orchestrate LLM agent workflows but treat scheduling as a local concern. Our model addresses the multi-agent, multi-user scheduling problem where agents compete for shared resources with different criticality levels.

Computing continuums. Dustdar et al. [1] survey the computing continuum vision. Our work contributes a concrete scheduling model for the heterogeneous workloads that arise in these environments.

IX. Conclusion

We presented eight diverse workload use cases in computing continuums—including self-modifying agentic AI workflows and real-time IoT with preemption—and demonstrated that single-integer priority is insufficient for scheduling heterogeneous DAG workloads. Through systematic analysis, we identified six requirements and showed that no existing method satisfies all six.

Our hybrid priority model combines lexicographic urgency classes with weighted multi-dimensional scoring. The urgency classes provide auditable structural guarantees for safety-critical work and serve as triggers for preemption on resource-constrained executors. The scoring function provides expressive, domain-specific ordering within each class, including novel dimensions such as agent iteration depth decay for agentic AI workloads. Fairness constraints prevent starvation independently of priority scores.

The model subsumes classical scheduling policies as special cases, yet each component is independently configurable and interpretable. We identified inter-workflow dependencies, failure recovery, and adaptive agent decay as open problems. Future work includes implementation in ColonyOS, empirical evaluation with concurrent workloads, learned weight tuning, and formal analysis of preemption overhead on edge deployments.

References

[1] S. Dustdar, V. Casamayor Pujol, and P. K. Donta, "On distributed computing continuum systems," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4092–4105, 2023.

[2] ColonyOS, "ColonyOS – Distributed meta-orchestrator," 2024. [Online]. Available: https://github.com/colonyos/colonies

[3] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Comput. Surv.*, vol. 31, no. 4, pp. 406–471, 1999.

[4] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.

[5] T. D. Braun *et al.*, "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," *J. Parallel Distrib. Comput.*, vol. 61, no. 6, pp. 810–837, 2001.

[6] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant resource fairness: Fair allocation of multiple resource types," in *Proc. NSDI*, 2011, pp. 323–336.

[7] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. HotNets*, 2016, pp. 50–56.

[8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proc. SIGCOMM*, 2019, pp. 270–288.

[9] V. K. Vavilapalli *et al.*, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. SoCC*, 2013, pp. 1–16.

[10] S. Vestal, "Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance," in *Proc. RTSS*, 2007, pp. 239–243.

[11] A. Burns and R. I. Davis, "A survey of research into mixed criticality systems," *ACM Comput. Surv.*, vol. 50, no. 6, pp. 1–37, 2019.

[12] Kubernetes, "Pod priority and preemption," 2024. [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/